Theoretical Foundations of Artificial Intelligence

Unit IV – Reasoning Under Uncertainty

1. Why uncertainty?
2. Introduction to Non-Monotonic Reasoning
3. Logics for Non-Monotonic Reasoning
4. DFS with Dependency-directed backtracking
5. Justification based Truth Maintenance Systems
6. Statistical Reasoning: Bayes Theorem for probabilistic inference
7. Certainty Factors and Rule-Based Systems
8. Bayesian Belief Network
9. Dempster-Shafer Theory
10. Fuzzy Logic

UNIT IV: REASONING UNDER UNCERTAINTY

1. What is Reasoning Under Un-certainty

In real life, information is **rarely complete or perfectly certain**.
Yet AI systems must still make **decisions or inferences** from **partial, noisy, or conflicting data**.

Example1: Imagine you're a doctor diagnosing a patient:
You don't have full certainty — symptoms could mean many things.

- Fever + cough could mean **flu**, **COVID**, or **pneumonia**.
  You *reason under uncertainty* to find the **most probable cause**.

Example2: Think of Google Maps:

- It predicts the *best route*, even though traffic data is **partially outdated**.
- It *estimates*, not guarantees.

Example3: Self-Driving Cars:

Sensors detect a "pedestrian" ahead, but fog makes it uncertain—is it a person, a shadow, or a plastic bag? The car slows down probabilistically, avoiding accidents.

Thus, **AI systems need reasoning mechanisms** that can handle **incomplete, inconsistent, or probabilistic knowledge** — just like humans do.

Why Uncertainty exits

Uncertainty arises because:

1. **Incomplete data** — we don't have all facts.
   A simple rule might be: IF cough AND fever THEN flu. The system concludes "flu" and stops. But what if it's just a common cold? Or COVID? Or allergies? The real world is messy. Symptoms overlap, tests are imperfect, and knowledge is incomplete.

2. **Noisy data** — sensors give errors.
   Example: A camera misreads a color.

If a rule says: IF smoke is seen THEN fire is present

This works most of the time — but not always (it could be fog, or dust).
So, AI must handle **uncertainty of inference**, not just **truth or falsehood**.

2. Introduction to Non-Monotonic Reasoning

Core Idea: A reasoning system where **adding new knowledge can invalidate previous conclusions**. It's the opposite of traditional monotonic logic. It allows systems to make "best guess" assumptions in the absence of complete information.

Monotonic vs Non-Monotonic:

Monotonic: Adding new information never invalidates previous conclusions
(e.g., Mathematics — once proven, always true)

Non-Monotonic: New information can invalidate previous conclusions

You assume:

Birds can fly → Therefore, Tweety can fly.

But when you learn:

Tweety is a penguin.

You retract your conclusion.
This “changing of belief” is **non-monotonic reasoning**.

3. Logics for Non-Monotonic Reasoning

These are formal logical systems designed to handle non-monotonicity.

The Big Idea: "Best Guess" Logic

Imagine your brain has a rule: **"Make the best guess you can with the information you have, and be ready to change your mind if you learn something new."**

That's what Non-Monotonic Reasoning is all about. "Non-Monotonic" is a fancy word for **"My conclusions can go down."** In normal math, if you have 5 apples and add 2, you have 7. The number only goes up. But in this kind of thinking, your answer of "7 apples" could change back to "5" if you learn that the 2 new apples were actually made of plastic!

The "logics" are just different sets of rules for making and changing these best guesses.

Default Logic:

Rules with exceptions

Format: "Normally P, unless exceptional circumstances"

Example 1: The Mysterious Backyard Creature

You look out the window and see a small, furry animal with a bushy tail running along the fence.

- **Your Brain's Default Logic Rule:** IF something is small, furry, and has a bushy tail, THEN it is probably a squirrel.
- **Your Conclusion:** "It's a squirrel!"

This is your **best guess**. You assumed it was a normal backyard animal.

Now, you get new information: The animal stops and lets out a loud "CH-CH-CH" sound and has a black mask across its eyes.

- **Your Brain's New Rule Kicks In:** IF something has a black mask and makes a "chattering" sound, THEN it is a raccoon.
- **Your Updated Conclusion:** "Wait, it's not a squirrel! It's a raccoon!"

What Happened?
You used **Default Logic**. You made a default assumption ("it's a squirrel") because you had no reason to think otherwise. When new evidence came in, you changed your mind. Your first conclusion was *defeated* by the new fact.

Circumscription:

A formal way of saying ***"assume everything is as normal as possible unless stated otherwise."*** It "minimizes" abnormalities. We assume Tweety is a "normal" bird (one that flies) until we are forced to categorize him as an "abnormal" bird (a non-flyer).

Example: The "Finished" Homework
You tell your friend: "I finished all my homework."

- **Your Friend's Circumscription Logic Rule:** They assume everything is normal and typical unless they hear otherwise. Their rule is: IF someone says they finished their homework, THEN assume they finished it correctly and on paper (not eaten by the dog).
- **Their Conclusion:** "Great! You're all done."
This is the **simplest, most normal conclusion**.

Now, you get new information: You open your backpack and say, "Oh no! My dog actually chewed up my math worksheet!"

- **Your Friend's Logic Updates:** The "normal" assumption has been broken. An abnormality (dog-eating-homework) has occurred.
- **Their Updated Conclusion:** "Oh, so you're *not* actually finished. You have to redo it."

What Happened?
Your friend used a kind of logic called **Circumscription**. They *minimized abnormalities*—they assumed your homework situation was normal and problem-free. **When they learned about the abnormal dog event, they had to take back their first conclusion.**

Stable Models / Autoepistemic Logics:

Reasoning about an agent's own knowledge or belief.

The Big Idea: This is when the AI thinks about its **own knowledge**. The word "autoepistemic" just means "self-knowledge" - ***knowing what you know and what you don't know.***

Example: The Party Planning
Your friend says: "If I don't **know** that Sarah is coming to my party, then I won't order her favorite pizza."

- **The AI's Rule:** IF I don't know Sarah is coming, THEN don't order veggie pizza.
- **What Happens:** The AI checks: "Do I know Sarah is coming? No. Therefore, I'll just order pepperoni."
  Later, Sarah texts: "I'm coming!"

- Now the AI **knows** Sarah is coming.
- The condition changes, so the conclusion changes.

New Decision: "I should order some veggie pizza too!"

Default Logic, **Circumscription**, and **Autoepistemic Logic** all try to handle **reasoning with incomplete information**, but they do it in *different ways*.

Default Logic Vs Circumscription Vs Autoepistemic Logic

Default Logic – *"Assume normally, unless told otherwise"*

Example1:
"If something is a bird, then by default, assume it can fly."

Example2:
You see a person wearing a white coat → you **assume they're a doctor** (default belief).
But if someone says "He's actually a painter," → you withdraw your assumption.

Key Point:
Default Logic focuses on **rules with exceptions** and **default assumptions**.

Circumscription – *"Assume the world is as small and simple as possible"*

You minimize the number of things that are *abnormal* or *exceptions*.

Example:
Imagine a detective investigating a crime.
He assumes **only the fewest number of people necessary** were involved — he won't imagine extra suspects unless the evidence demands it.

Key Point:
Circumscription is about **minimizing abnormalities** — "the world is normal unless proven otherwise."

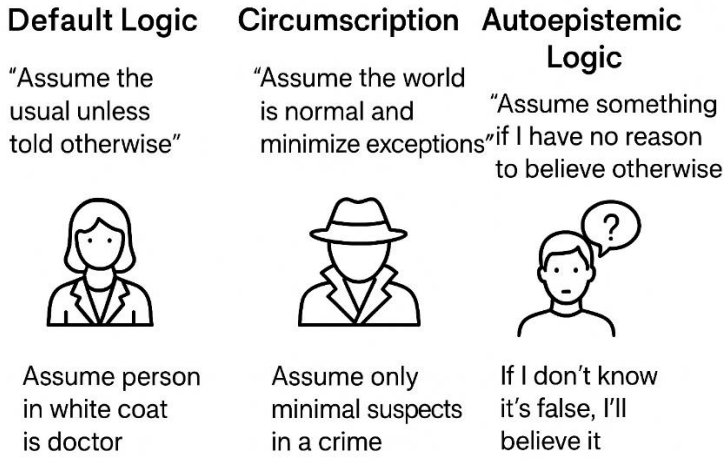
Autoepistemic Logic – *"Reason about what I know and don't know"*

This logic is about **self-knowledge** — what an agent **believes**, **doesn't believe**, or **cannot prove**.

Example1:

"If I don't know that Ostrich can't fly, I'll believe Ostrich can fly."

Example2:
You're deciding whether to carry an umbrella.
You think: "If I don't know that it won't rain, I'd better take it."
You're reasoning based on **your own state of knowledge**, not just the facts.



4.DFS with Dependency-Directed Backtracking

Problem:
In **rule-based systems** or **search problems**, a wrong assumption can lead to dead-ends.
Ordinary **Depth First Search (DFS)** backtracks *blindly*, step-by-step.

Dependency-directed backtracking:
Instead of going one step back, it jumps **directly to the source of the wrong assumption** (the cause of failure).

In **dependency-directed backtracking**, the system keeps track of **why** each decision was made — i.e., **dependencies or assumptions** that led to it.

When a failure happens, instead of going back step-by-step,
the algorithm uses this dependency record to **"jump directly"** to the **real cause** of the problem.

How it does that:

- Each choice or variable stores its **justification** — a list of earlier assumptions that influenced it.
- When a conflict occurs, the system looks at this justification list to find **which assumption(s)** actually caused the conflict.
- Then it **jumps directly** to the **earliest assumption** in that list and changes it — skipping all the irrelevant steps in between.

Example: A Detective Solving a Case

Let's say you are a detective connecting clues (A → B → C → D).

- You assume A = "the servant did it."
- B = "the servant had the key."
- C = "the servant was at the crime scene."

- D = “the murder weapon had his fingerprints.”

Now suddenly, you find **new evidence**: the fingerprints actually belong to someone else → conflict!

In **normal backtracking**, you’d undo D, then C, then B, then A — checking everything again.
That’s **slow**.

But in **dependency-directed backtracking**, you know:
“D depended only on the assumption that the butler had the key (B).”

So, instead of undoing C and A unnecessarily, you **jump directly back to B**, change that assumption, and continue reasoning forward.

So, you jump directly to the cause of the contradiction, not just the last step.

5. Truth Maintenance Systems

Purpose:

- Keep track of beliefs and their justifications
- Handle contradictions when they arise
- Update beliefs when assumptions change

Types:

- Justification-based (JTMS): Track support for beliefs
- Assumption-based (ATMS): Track different contexts

Justification-Based Truth Maintenance Systems (JTMS)

A JTMS holds **one set of beliefs**. It carefully tracks the reasons for each belief. If one reason is proven wrong, it has to **retract all beliefs that depended on it** and figure out a new, single consistent story. It's a bit "all-or-nothing."

Example: A detective(s) can only keep **one main theory** in their head at a time. They are very focused but have to change everything if new evidence proves them wrong.

Let's follow the JTMS Detective:

1. **Step 1:** They see the broken window (Clue 1). Using **Rule A**, they believe: "It was a break-in."
2. **Step 2:** They see the muddy footprints (Clue 2). Using **Rule B**, they believe: "The suspect came from the garden."

3. **Step 3:** Now they have "break-in" AND "from the garden." Using **Rule C**, they conclude their main theory: **"The Gardener did it!"**

Their notebook (the TMS) looks like this:

- Belief: Gardener is guilty.
- Justification: Because of Rule C (break-in + from the garden).

4. **The Plot Twist!** The homeowner walks in and says, "Oh, I broke that window yesterday playing baseball inside. I haven't fixed it yet."
5. **What happens?** The foundation of the theory crumbles! The "break-in" belief is now **false**.
    - The JTMS detective goes into action. They erase "break-in" from their notebook.
    - This automatically makes the "Gardener is guilty" belief invalid because its justification is gone.
    - The detective's entire theory collapses. They are back to square one and have to start over with the remaining clues.

Assumption-Based Truth Maintenance System (ATMS)

An ATMS explores **multiple belief sets** simultaneously. It works with assumptions and figures out the consequences of different combinations. When new information arrives, it doesn't rebuild from scratch; it just throws away the specific scenarios that are no longer possible. It's great for exploring "what-if" situations.

It helps **rule-based AI systems** reason efficiently when there are **alternative or competing hypotheses** — without discarding useful assumptions.

Example: This detective is more imaginative. They don't just follow one theory; they explore **all possible theories at the same time**. They keep a separate file for each possible scenario.

How they work:

- They treat some things as **assumptions** (educated guesses), not certain facts.
- They then see what beliefs each set of assumptions leads to.
- They keep track of **all possible worlds** that are consistent with the clues.

Let's follow the ATMS Detective:

1. **Step 1:** They see the clues but don't commit to one story. Instead, they label things.
    - They say, "Let's **assume** it was a break-in." (Assumption A)
    - "Let's **assume** the suspect came from the garden." (Assumption B)
    - "Let's **assume** the Butler went to the bakery." (Assumption C)

2. **Step 2:** They build multiple case files (called **environments**).
    - **Case File 1:** {Assumption A, Assumption B}
        - Using **Rule C**, this file concludes: **"Gardener is guilty."**
    - **Case File 2:** {Assumption C}
        - Using **Rule D**, this file concludes: "Butler might be a suspect." (But it's not a strong case yet).
    - **Case File 3:** {Assumption A, Assumption C} ...and so on.

3. **The Same Plot Twist!** The homeowner says, "I broke that window myself."

4. **What happens?** The ATMS detective doesn't panic!
    - They simply go to their stack of case files and **marks any file that used Assumption A as "INVALID."**
    - **Case File 1** ({Assumption A, Assumption B}) is thrown in the trash.
    - **Case File 2** ({Assumption C}) is still perfectly valid! It never assumed a break-in.
    - The detective didn't lose any work. They still have all their other theories intact and can now focus on them.

The Big Difference (Like Comparing Detective Notebooks)

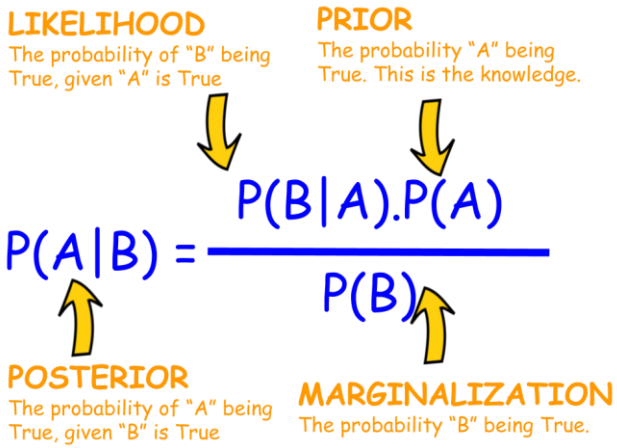
- **JTMS is like a single detective's linear report:** One main belief at a time, backed by clear "why" reasons. When something breaks, you fix that one chain (fast for simple stuff, but restarts big if everything's connected). It's all about keeping your *current story* clean and justified.

- **ATMS is like a detective's conspiracy board with yarn and pins:** Multiple stories running side-by-side, based on "what if" assumptions. It explores *all possible outcomes* without committing, so you can swap pieces easily when evidence changes. It's more powerful for complex cases but can get messy with too many threads.

Feature	JTMS	ATMS
Focus	Maintain one consistent belief set	Maintain many possible belief sets
Belief representation	Each belief has justifications	Each belief labelled with supporting assumptions
When contradiction occurs	Retract conflicting beliefs	Remove only inconsistent assumption sets
Memory	Keeps only current consistent knowledge	Keeps all possible alternatives
Efficiency	Simpler, faster	More complex, powerful
Analogy	Erase and rewrite	Label and filter scenarios
Use Case	When you need one clear, current model	When you need to explore multiple hypotheses

6. Statistical Reasoning with Bayes' Theorem

Bayes' Theorem



Prior (belief before seeing evidence)
P(A)

- Means: the probability of the hypothesis before seeing any evidence.

Likelihood (how probable the evidence is given the hypothesis)
P(B|A)

- Means: the probability of seeing the word "free" **if** the email is indeed spam.

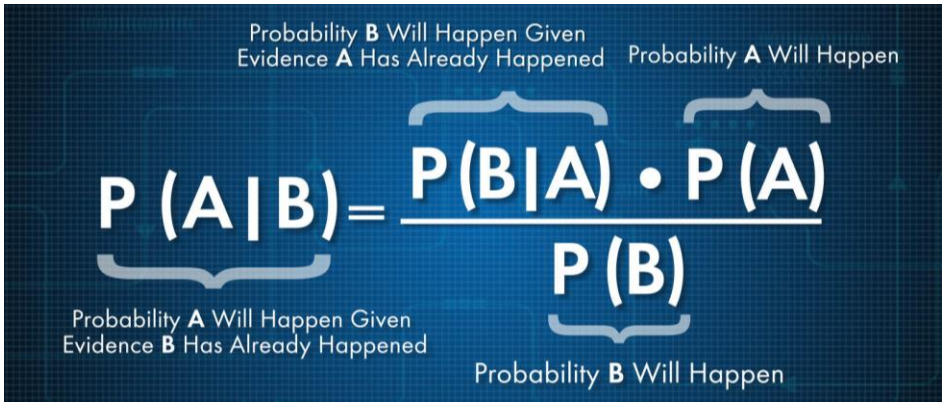
Marginalization (probability of the evidence)
P(B)

- Means: overall probability of seeing the word "free" in *any* email — spam or not.

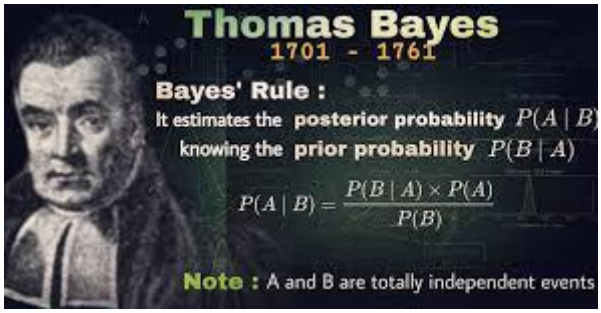
Posterior (updated belief after seeing evidence)
P(A|B)

- Means: probability that the email is spam **given** that it contains "free".

(OR)



Bayes' Theorem, which was first proposed by **Reverend Thomas Bayes (1701–1761)** — an English statistician, philosopher, and minister.



He developed a method to calculate the **probability of a cause given an observed effect** — something no one had formalized before.

If we take an example of email spam detection:

The formula can be converted to:

Symbol	Meaning in spam example
A	Email is **Spam**
B	Email contains the word "**free**"

So the formula becomes:

$$P(\text{Spam} \mid \text{“free”}) = \frac{P(\text{“free”} \mid \text{Spam}) \times P(\text{Spam})}{P(\text{“free”})}$$

1 Left side → Posterior Probability

$$P(\text{Spam} \mid \text{“free”})$$

👉 "Given that an email contains the word *free*, what is the probability that it's spam?"

This is the **final answer** we want — it tells us how confident we are that the email is spam **after** seeing the evidence ("free").

2 Numerator → How likely that word appears in spam

$$P(\text{“free”} \mid \text{Spam}) \times P(\text{Spam})$$

👉 Think of this as:

> "How likely is it that the email is spam **and** it contains 'free'?"

3️⃣ **Denominator → How common the word "free" is overall**

$$P(\text{“free”})$$

👉 This means:

> "In *all* emails (spam and not spam), how often does the word 'free' appear?"

Step1.

Let **S** = email is **Spam**.
Let **F** = email contains the word **"free"**.

We want:

$$P(S \mid F) \quad = \text{probability the email is spam given it contains “free”.}$$

Bayes' formula:

$$P(S \mid F) = \frac{P(F \mid S)\,P(S)}{P(F)}.$$

Step2. Pick simple, realistic numbers (from past data)

(You can replace these with values measured from your mailbox.)

- Prior probability an email is spam:
  $P(S) = 0.20$ (20% of emails are spam).
- Likelihood that spam contains "free":
  $P(F \mid S) = 0.40$ (40% of spam messages contain "free").
- Likelihood that legitimate mail (ham) contains "free":
  $P(F \mid \neg S) = 0.01$ (1% of ham messages contain "free").

Then $P(\neg S) = 1 - P(S) = 1 - 0.20 = 0.80$.

Step3. Compute the numerator P(F|S) P(S)

$$P(F \mid S)\,P(S) = 0.40 \times 0.20.$$

Digit-by-digit multiplication:

- $0.40 \times 0.20 = 0.080$.

(Think: $40 \times 20 = 800$, move decimal places: three places → 0.080.)

So numerator = **0.080**.

Step4. Compute P(F)— the denominator

The following formula is known as "Law of Total Probability"

$$P(F) = P(F \mid S)P(S) + P(F \mid \neg S)P(\neg S).$$

We already have the first part $= 0.080$. Compute the second part:

$$P(F \mid \neg S)P(\neg S) = 0.01 \times 0.80.$$

Multiply:

- $0.01 \times 0.80 = 0.008$.

Now add:

- $0.080 + 0.008 = 0.088$.

So $P(F) =$ **0.088**.

Note:
Why can't we take P(F) directly like P(S)?

Because:

- $P(S)$ is *given* directly by data ("20% emails are spam").
- $P(F)$ is *not directly given*; it depends on how "free" is distributed across both spam and non-spam emails.
- So we must **calculate it indirectly** using the law of total probability.

Let's understand this with an example:

Imagine you want to know: "What's the chance a random person wears glasses?"

But you only know:

- 60% of teachers wear glasses.
- 20% of students wear glasses.
- 10% of people are teachers, 90% are students.

You can't say "P(glasses) = 60% or 20%" directly.
You must **combine** both groups based on how big each group is:

Let:

- T = person is a **Teacher**
- S = person is a **Student**
- G = person **wears glasses**

Then, by the **Law of Total Probability**:

$$P(G) = P(G|T)P(T) + P(G|S)P(S)$$

(OR)

$$P(G) = P(G|T)P(T) + P(G|\neg T)P(\neg T)$$

So, in place of saying "Student" we can also say "Not Teacher"

Substituting given values

$$P(G) = (0.6)(0.1) + (0.2)(0.9)$$

$$P(G) = 0.06 + 0.18 = 0.24$$

Final Answer:

$$\boxed{P(G) = P(G|T)P(T) + P(G|S)P(S) = 0.24}$$

So, the **Interpretation** should be**:**

There's a **24% chance** that a randomly chosen person wears glasses — considering both teachers and students in the population.

Step5. Plug into Bayes' formula

$$P(S \mid F) = \frac{0.080}{0.088}.$$

To make division easier, clear decimals: multiply numerator & denominator by 1000:

$$\frac{0.080}{0.088} = \frac{80}{88}.$$

Simplify fraction: divide top and bottom by 8:

$$\frac{80}{88} = \frac{10}{11}.$$

Now do the long division $10 \div 11$ digit-by-digit to get a decimal:

- 11 into 10 → 0 (decimal point), remainder 10.
- Append 0 → 100. 100 ÷ 11 = 9 (9×11 = 99). Remainder = 100 − 99 = 1.
- Append 0 → 10. 10 ÷ 11 = 0 (0×11 = 0). Remainder = 10.
- Append 0 → 100. Repeat: 100 ÷ 11 = 9, remainder 1.
- The pattern 09 repeats.

So $10/11 = 0.909090\ldots$ (repeating).

Therefore:

$$P(S \mid F) \approx \mathbf{0.9091} \quad (\text{about } 90.91\%).$$

Exercise: Find the Probability of an email being spam if it contains the word "free"

We have the following information:

Category	Total Emails	Emails with "free"
Spam	200	100
Not Spam	800	40
Total	1,000	140

Now, let's calculate each part of Bayes' Theorem step by step. We'll define:

- **S** = Email is spam.
- **F** = Email contains the word "free".
- Goal: Find **P(Spam | "free")** which is **P(S|F)** (probability it's spam *given* "free" appears).

Step 1: Calculate P(S) – Overall Probability of Spam

This is just how common spam is in our dataset. P(S) = Number of spam emails / Total emails = 200 / 1,000 = **0.2** (or 20%). *Explanation*: Out of every 10 emails, 2 are spam on average. This is our "prior belief" before seeing any words.

Step 2: Calculate P(F|S) – Probability of "Free" *If* It's Spam

This measures how spammy the word "free" is (how often it shows up in spam). P(F|S) = Spam emails with "free" / Total spam emails = 100 / 200 = **0.5** (or 50%). *Explanation*: Half of all spam emails use "free" – it's a strong clue! (In data driven AI (Practical AI), this is learned from examples/dataset.)

Step 3: Calculate P(F) – Overall Probability of "Free"

This is the total chance of seeing "free" in *any* email (spam or not).

We get it using the law of total probability:

P(F) = [P(F|S) × P(S)] + [P(F|Not S) × P(Not S)]

First, we need P(F|Not S): P(F|Not S) = Non-spam emails with "free" / Total non-spam = 40 / 800 = **0.05** (or 5%).

Note: "Free" is rare in legit emails (e.g., a newsletter might say it occasionally).

Now plug in: P(F) = (0.5 × 0.2) + (0.05 × 0.8) = (0.1) + (0.04) = **0.14** (or 14%).

So: 14% of all emails have "free" – mostly from spam, but some from good emails.

Step 4: Put It All Together – Calculate P(S|F)

Now use Bayes' Theorem:

P(S|F) = [P(F|S) × P(S)] / P(F)

= (0.5 × 0.2) / 0.14

= 0.1 / 0.14

≈ **0.714** (or 71.4%).

Assignment: Take an example of Medical Diagnosis Domain for finding the probability of Lung Damage based on Smoking and Not Smoking of Patients

With the following data:

Category	Total number of patients	Patients with “Smoking” habit
Lung Damage	100	70
No Lung Damage	900	20
Total	1000	90

Applications of Bayes’ theorem

Field	Example Use	What Bayes’ Theorem Does
Email Filtering	Spam Detection	Updates spam probability based on words
Medicine	Disease Testing	Adjusts diagnosis probability
Law	Forensic Evidence	Updates guilt probability
AI	Naïve Bayes Classifier	Predicts categories – hotel review positive or negative
Finance	Credit Risk	Predicts likelihood of default (loan will be repaid or not)
Weather	Rain Forecast	Updates weather predictions

7. Certainty Factors and Rule Based Systems

“**Certainty Factors (CFs)**” is one of the *most practical and intuitive* approaches used in **Reasoning Under Uncertainty**, especially in early **Rule-Based Expert Systems** like **MYCIN** (the famous medical diagnosis AI from the 1970s).

The core problem while dealing with Un-Certainty

In a perfect world, all our knowledge would be absolute. A rule like IF the engine is silent THEN the car has a dead battery would always be 100% true. But in the real world, this is rarely the case.

- Maybe the engine is silent because it's out of gas.
- Maybe the starter motor is broken.
- Maybe the battery is just weak, not completely dead.

How does a rule-based system reason with this kind of uncertain, heuristic knowledge? This is where **Certainty Factors** come in.

What are Certainty Factors

A **Certainty Factor (CF)** is a numerical value that represents the ***degree of belief or confidence*** in a fact or a rule. It was developed in the 1970s for the **MYCIN** expert system, which diagnosed bacterial infections.

The key idea is to separate the *measure of belief* from the *measure of disbelief*.

CF Range: Typically, from -1 to +1.

- CF = +1.0: Absolute belief (It is definitely true).
- CF = 0.0: Complete ignorance (We have no information).

- CF = -1.0: Absolute disbelief (It is definitely false).

Example: A Vehicle diagnosis System

Requirement: Let's diagnose why a car won't start.

The power of CFs comes from the formulas used to combine them as the inference engine chains rules together.

Rules:

1. `IF engine_is_silent THEN dead_battery (CF=0.7)`
2. `IF lights_are_dim THEN dead_battery (CF=0.6)`
3. `IF fuel_gauge_is_empty THEN out_of_gas (CF=0.9)`

User Input / Sensor Data:

- `engine_is_silent (CF=0.9)` // Very sure the engine didn't make a sound.
- `lights_are_dim (CF=0.3)` // Slightly sure the lights are dim.
- `fuel_gauge_is_empty (CF=0.1)` // The gauge is acting a bit strange, low confidence.

Inference Process:

1. **Rule 1 Fires:**
   - Premise CF = `0.9`
   - Conclusion: `CF(dead_battery) = 0.9 * 0.7 = 0.63`
2. **Rule 2 Fires:**
   - Premise CF = `0.3`
   - Conclusion: `CF(dead_battery) = 0.3 * 0.6 = 0.18`
3. **Combine Evidence for `dead_battery`:**
   - `CF_combined = 0.63 + 0.18 - (0.63 * 0.18)`
   - `= 0.81 - 0.1134`
   - `= 0.6966`
4. **Rule 3 Fires:**
   - Premise CF = `0.1`
   - Conclusion: `CF(out_of_gas) = 0.1 * 0.9 = 0.09`

Final Diagnosis:

- `dead_battery` with a certainty of `0.70`
- `out_of_gas` with a certainty of `0.09`

The system can now confidently suggest, "The most likely problem is a dead battery."

Example2: Medical Diagnosis Example

This medical example shows how certainty factors can help healthcare professionals weigh evidence from multiple symptoms to form differential diagnoses with quantified confidence levels, much like how a mechanic diagnoses car problems!

The Rules:

1. **IF** patient_has_fever **THEN** viral_infection (CF=0.8)
2. **IF** patient_has_nausea **THEN** migraine (CF=0.7)
3. **IF** patient_has_fever **AND** patient_has_rash **THEN** measles (CF=0.9)
4. **IF** patient_has_stress **THEN** tension_headache (CF=0.6)
5. **IF** patient_has_vision_changes **THEN** migraine (CF=0.8)

Patient Symptoms (with Certainty Factors):

- `patient_has_fever` (CF=0.9) // High fever confirmed by thermometer
- `patient_has_nausea` (CF=0.4) // Mild nausea reported
- `patient_has_rash` (CF=0.7) // Visible rash but not severe
- `patient_has_stress` (CF=0.6) // Patient reports work stress
- `patient_has_vision_changes` (CF=0.3) // Slight blurriness occasionally

Measles Rashes

Inference Process with Certainty Factors

Step 1: Apply Individual Rules

Rule 1: `IF fever THEN viral_infection (CF=0.8)`

- Premise CF = 0.9
- CF(viral_infection) = 0.9 × 0.8 = **0.72**

Rule 2: `IF nausea THEN migraine (CF=0.7)`

- Premise CF = 0.4
- CF(migraine) = 0.4 × 0.7 = **0.28**

Rule 3: `IF fever AND rash THEN measles (CF=0.9)`

- Premise CF = min(0.9, 0.7) = 0.7
- CF(measles) = 0.7 × 0.9 = **0.63**

Rule 4: `IF stress THEN tension_headache (CF=0.6)`

- Premise CF = 0.6
- CF(tension_headache) = 0.6 × 0.6 = **0.36**

Rule 5: `IF vision_changes THEN migraine (CF=0.8)`

- Premise CF = 0.3
- CF(migraine) = 0.3 × 0.8 = **0.24**

Step 2: Combine Evidence for Same Conclusions

We have two rules suggesting **migraine** (from Rule 2 and Rule 5):

Combine CFs for migraine:

- CF_1(migraine) = 0.28 (from nausea)
- CF_2(migraine) = 0.24 (from vision changes)

Using the combination formula:

```
CF_combined = CF1 + CF2 - (CF1 × CF2)
CF_migraine = 0.28 + 0.24 - (0.28 × 0.24)
CF_migraine = 0.52 - 0.0672
CF_migraine = **0.4528**
```

Step 3: Final Diagnosis with Certainties

After combining all evidence:

- **viral_infection**: CF = **0.72**
- **measles**: CF = **0.63**
- **migraine**: CF = **0.45**
- **tension_headache**: CF = **0.36**

Note: When there are 3 or more rules for the same conclusion

You simply **apply the formula iteratively** (step by step).

That means:

1. Combine the first two:

$$CF_{12} = CF_1 + CF_2 - (CF_1 \times CF_2)$$

2. Then combine the result with the third:

$$CF_{123} = CF_{12} + CF_3 - (CF_{12} \times CF_3)$$

3. If you have more, you just keep going in the same pattern.

8. Bayesian Belief Networks (Bayes Nets)

The core idea is combine multiple uncertain causes to estimate most likely outcome.

Bayesian Belief Networks create a **cause-and-effect map with probabilities**, allowing you to update your beliefs about unknown things when you learn new information.

Judea Pearl (Computer Scientist, UCLA), created the **Bayesian Belief Network** (also called *Bayes Net* or *Belief Network*). He extended Bayes' theorem to handle **multiple uncertain variables** and their **dependencies**.

Judea Pearl

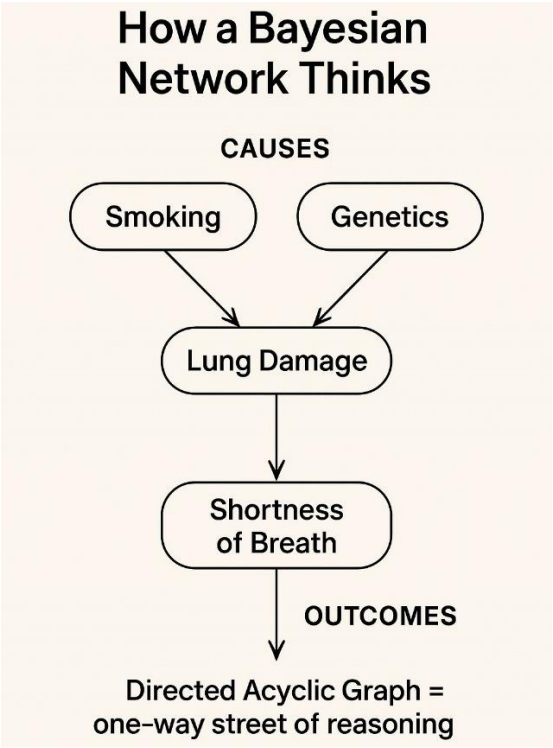
- Expanded Bayes' single equation into an entire **graphical model** connecting many variables.
- Introduced a way to represent **conditional dependencies** visually and computationally.

- His networks could reason like humans — "If fever increases, flu becomes likely, but cold might also explain it."

This was revolutionary for **AI reasoning under uncertainty**.

Judea Pearl won the **Turing Award (2011)** — often called the "Nobel Prize of Computing" — for his work on **Bayesian networks** and **causal reasoning**.

Structure of a Bayesian Belief Network with an example:



Directed: Arrows have a direction (cause → effect).
Acyclic: No loops — you can't go in circles (you can't end up back at Smoking).
Graph: The structure of nodes and arrows connecting them.

How Bayesian Reasoning Works Here:

Eg:

- $P(\text{Lung Damage}|\text{Smoking, Genetics})$:
  probability of lung damage given smoking and genetics.
- $P(\text{Shortness of Breath}|\text{Lung Damage})$:
  probability of shortness of breath given lung damage.

So instead of storing all combinations of data, the Bayesian network breaks them into small, local relationships — making reasoning efficient and structured.

Conditional Probability Table (CPT)

In a **Bayesian Belief Network**, each **node (variable)** has a **Conditional Probability Table (CPT)** that tells us **how likely** that node is to be true (or take a value) depending on the states of its **parent nodes** (the causes feeding into it).

Think of It Like a "Rulebook of Chances"

Imagine every node (like *Lung Damage*) keeps a **small rulebook (from expert judgement/calculated from statistics provided we have related dataset)**:

"If Smoking = Yes and Genetics = Yes → 90% chance of Lung Damage."
"If Smoking = Yes and Genetics = No → 70% chance of Lung Damage."
"If Smoking = No and Genetics = Yes → 40% chance of Lung Damage."
"If Smoking = No and Genetics = No → 10% chance of Lung Damage."

That rulebook **is the Conditional Probability Table**.

Condition Probability Table for the node LungDamage:

It defines:

$$P(LungDamage = True \mid Smoking, Genetics)$$

Smoking	Genetics	P(LungDamage = True)
Yes	Yes	0.9
Yes	No	0.7
No	Yes	0.4
No	No	0.1

It shows how this node's probability changes **based on the conditions** of its parent nodes (Smoking and Genetics).

What about Smoking and Genetics?

They have **no parents**, so their probabilities are just **Prior Probabilities**, not CPTs.

Variable	P(True)	P(False)
Smoking	0.3	0.7
Genetics	0.2	0.8

Note: A **CPT** exists for every node that has at least one parent.
It tells us: ***"Given my parents' states, what is my probability of being True (or each possible value)?"***

Let's assume the following probabilities:

Person Smokes: 0.3
So, Person doesn't smoke: 0.7

Person has risky genes: 0.2
So, Person has no risky genes: 0.8

Smoking	Genetics	P(S)	P(G)	P(L | S, G)	**Joint Probability = P(S,G,L)**	Explanation
Yes	Yes	0.3	0.2	0.9	**0.3 × 0.2 × 0.9 = 0.054**	Smokes + bad genes → high lung risk
Yes	No	0.3	0.8	0.7	**0.3 × 0.8 × 0.7 = 0.168**	Smokes only → still high risk
No	Yes	0.7	0.2	0.4	**0.7 × 0.2 × 0.4 = 0.056**	No smoking, but genetic risk
No	No	0.7	0.8	0.1	**0.7 × 0.8 × 0.1 = 0.056**	No smoking, no genetics → low risk

Total Probability of Lung Damage

Add all joint probabilities (because they are all possible cases):

$$P(LungDamage = True) = 0.054 + 0.168 + 0.056 + 0.056 = 0.334$$

So, there's a 33.4% overall chance that a random person has lung damage, considering all combinations of smoking and genetics.

Advantages of (Bayesian Belief Network) BBN:

1. Represent Uncertainty Naturally
In real life, we rarely have "yes/no" certainty — instead, we have *degrees of belief*.
BBNs allow probabilities between 0 and 1 to represent such uncertain knowledge.

Example:
A doctor might not say "the patient *has* pneumonia," but instead,
"Given symptoms, there's a 75% chance of pneumonia."
This is exactly what BBNs capture.

2. Graphical + Probabilistic Representation
BBNs combine **graph theory** and **probability theory** — making complex systems easier to understand.

- Nodes represent variables.
- Arrows represent cause–effect relationships.
- CPTs quantify those relationships.

It's like a *map of belief dependencies* — easy to visualize and reason about.

3. Support Causal Reasoning
The directed arrows often represent *causal* relationships ("smoking → lung damage"),
so you can reason *both ways*:

- **Predictive:** Given cause → find effect.
- **Diagnostic:** Given effect → find possible cause.

Example:
If someone smokes → predict higher lung damage risk.
If lung damage detected → infer smoking or genetics might be the reason.

4. Allow Probabilistic Inference
BBNs can compute probabilities for unknown variables based on known evidence using **Bayes' theorem**.

Example:
If we know a patient has lung damage, the BBN can automatically estimate "What is the probability they smoke?" That's *inference* — done efficiently through the network.

5. Support for Decision Making and Risk Analysis
BBNs can be extended to **Decision Networks** or **Influence Diagrams**,
where they not only infer probabilities but also help **choose actions** based on expected outcomes.

Example:
In finance: "Should we approve this loan?"
→ depends on nodes like income, credit score, default probability, etc.

etc

Where Bayesian Networks Are Used Today

Domain	Example
Healthcare	Disease diagnosis (like flu, cancer prediction)
Finance	Credit risk, fraud probability
Autonomous Cars	Deciding if an obstacle is dangerous based on uncertain sensors
Robotics	Sensor fusion (combining uncertain data)
AI Planning	Predicting the effect of uncertain actions
Legal AI	Assessing likelihood of guilt based on evidence
Natural Language Processing	Word prediction and context disambiguation

RETE Network Vs Bayes Net

Key Idea	RETE	Bayesian
Logic Type	Rule-based (deterministic)	Probabilistic (uncertain)
Main Goal	Speed up rule execution	Model uncertainty and belief
Common Use	Expert systems like Drools, CLIPS	Predictive systems like spam filters, medical diagnosis

Bayesian Belief Network Vs ATMS

Bayesian Belief Networks (BBNs) and **Assumption-Based Truth Maintenance Systems (ATMS)** sound similar — both handle reasoning under uncertainty or incomplete knowledge —
but they come from **different roots** and handle uncertainty in **different ways**.

Aspect	Bayesian Belief Network (BBN)	Assumption-Based Truth Maintenance System (ATMS)
Type of Uncertainty	*Probabilistic uncertainty* — how *likely* something is true	*Logical uncertainty* — which *set of assumptions* make something true
Representation	Graph of variables (nodes) with conditional probabilities	Graph of assumptions and derived beliefs (justifications)
Computation Method	Uses **Bayes' Theorem** to update beliefs numerically	Uses **logical dependency tracking** to update truths symbolically
When new info arrives	Updates *belief probabilities* (e.g., "now 0.8 chance of flu")	Updates *truth consistency* (e.g., "this fact no longer holds under these assumptions")
Invented by	Judea Pearl (1980s)	Jon Doyle (1979–1983)
Field	Probabilistic AI	Symbolic AI / Rule-Based Systems

The Power of BBNs: Reasoning in All Directions

We can use this network to perform different types of reasoning.

1. Forward Reasoning (Predictive Reasoning)

- **Question:** "It is pollen season. What is the probability that the person is sneezing?"
- **We know:** `Season = Yes`
- **We want:** P(`Sneezing = Yes` | `Season = Yes`)

The network can chain the probabilities together to calculate this. (The math involves summing over all possibilities, but the network structure makes it efficient).

2. Backward Reasoning (Diagnostic Reasoning - Most Powerful!)

- **Question:** "I see the person is sneezing. What is the probability that it is pollen season?"
- **We know:** `Sneezing = Yes`
- **We want:** P(`Season = Yes` | `Sneezing = Yes`)

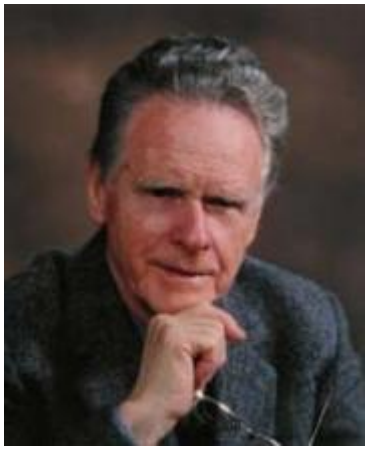
This is exactly like a medical diagnosis! We see a symptom (sneezing) and want to infer the cause (pollen season). This is done using **Bayes' Theorem**, and the BBN is perfectly designed to handle this calculation seamlessly.

9. Dempster-Shafer Theory (DST) (also known as Theory of Evidential Reasoning)

It's an **alternative to probability theory** for handling **uncertainty.**

Dempster–Shafer Theory is a **mathematical theory of evidence** used to deal with **uncertainty** — especially when we **don't have enough information** to assign precise probabilities like in Bayes' theorem.

It allows us to **combine evidence from different sources** to calculate the probability of events. It is particularly useful when information is incomplete or conflicting.



Dempster

Shafer

Where **Bayesian theory** requires *precise probabilities*,

Dempster–Shafer theory allows you to say
"I'm not sure — but I have *some belief* in one outcome and *some ignorance* about the rest."
Example: Imagine you're a detective investigating who stole a diamond

You have 3 suspects:

- A = Kiran
- B = Murali
- C = Vikram

You don't have full proof yet, but you gather **pieces of evidence**:

- Fingerprints near the scene → 60% sure it's either **Kiran or Murali**
- A witness saw someone *short* → 30% sure it's **Murali**

But you're **not sure about Vikram at all** — you simply *don't know*.

Traditional probability would force you to assign *exact probabilities* to each suspect.
Dempster–Shafer lets you say:

> "I believe 60% that (Kiran or Murali) did it,
> 30% that Murali did it,
> and I'll keep 10% as *unknown* (maybe Vikram, maybe something else)."

So — it separates **belief** from **uncertainty**.

Components of DST

- **Belief** (Bel)= What you are *confident* about based on some evidences.

- **Plausibility (PL)** = What *could* be true, given what you don't know. (what is possible).

  Ex: Murali's **plausibility** is higher than his belief, because the unknown 60% *might* include him.

  So, according the above example:
  Pl(Murali) = Bel(Murali) + part of "Kiran or Murali" that could include Murali
  Pl(Murali) = 0.3 + 0.6 = 0.9

"Murali could possibly be the thief, up to 90% chance."

- **Un-Certainty / Ignorance** = What remains *unknown* (between Bel and Pl).

  Uncertainty=Pl−Bel

- **Frame of Discernment:** All possible outcomes. This is the set of all possible, mutually exclusive hypotheses.
  Example: {Kiran, Murali, Vikram}

- **Mass Function or The Basic Belief Assignment (BBA):** How much belief is directly assigned to a set. This assigns a value between 0 and 1 to each subset in the power set, representing the degree of belief that the truth lies in that subset exactly. The sum of all masses must be 1.

  Example: m({Murali}) = 0.3, m({Kiran,Murali}) = 0.6, m({Uncertainity}) = 0.1

So DST lets you model:
"I believe a little, I suspect more, but I'm not totally sure."

Example: Consider a scenario in artificial intelligence (AI) where an AI system is tasked with **solving a murder mystery using Dempster–Shafer Theory**.

The Situation/The Problem:

The setting is a room with four individuals: A, B, C, and D. Suddenly, the lights go out, and upon their return:

- B is discovered dead, having been stabbed in the back with a knife.
- No one else entered or exited the room
- It is known that B did not commit suicide. So, the murderer must be A, C or D.

The objective is to identify the murderer.

Step1. Frame of Discernment Θ (big Theta)

In Dempster–Shafer Theory, the **Frame of Discernment (Θ)** is the set of all possible hypotheses.

$$\Theta = \{A, C, D\}$$

That means the murderer could be **A**, **C**, or **D** — one of them, or a combination.

Step2. Power Set of Θ

To address this challenge using Dempster–Shafer Theory, we can explore various possibilities:

1. **Possibility 1:** The murderer could be either A, C, or D.
2. **Possibility 2:** The murderer could be a combination of two individuals, such as A and C, C and D, or A and D.
3. **Possibility 3:** All three individuals, A, C, and D, might be involved in the crime.

4. **Possibility 4:** None of the individuals present in the room is the murderer.

DST doesn't only consider single possibilities, but **all subsets** of Θ (this represents every possible combination of suspects).

$$2^{\Theta} = \{\emptyset, \{A\}, \{C\}, \{D\}, \{A, C\}, \{A, D\}, \{C, D\}, \{A, C, D\}\}$$

There are $2^3 = 8$ subsets (including the empty set).

- `{A}` → only A is the killer
- `{A, C}` → A or C could be the killer (we're unsure which one)
- `{A, C, D}` → total uncertainty (anyone could be)

Note: For example: {A,C} indicates → either A or C or both combined.

Step3. Assigning Basic Probability Mass (m)

Each subset gets a **basic probability assignment (BPA)** or **mass** m(X).
This expresses our **belief directly assigned** to that specific subset.

Let's assume two **sources of evidence**:

Evidence Source	Description	Mass Assignments
E_1	Fingerprints found near the body suggest A or C	$m_1(\{A, C\}) = 0.6$; $m_1(\{A, C, D\}) = 0.4$
E_2	Witness heard D and B arguing loudly	$m_2(\{D\}) = 0.7$; $m_2(\{A, C, D\}) = 0.3$

Note:

In each evidence, the first subset is belief and the 2nd subset is uncertainty.

Note: Each mass assignment represents:

For example: for Evidence E1:

- **$m_1(\{A, C\}) = 0.6$** → means *we believe with 60% confidence that the truth lies within {A, C}, but we don't know which one*.
- **$m_1(\{A, C, D\}) = 0.4$** → means *there's 40% belief in total uncertainty (anyone among A, C, D)*.

These are **not random** — they come from **evidence interpretation** (for example, fingerprints matching two people, witness statements, etc.).

We don't assign mass to:
- {A} or {C} → because E_1 never said *exactly A* or *exactly C*.
- {C, D}, {A, D} → because E_1 never connected D with anyone else.

Note: **in Dempster–Shafer theory, we assume each evidence source is *independent***.

That's what allows us to **combine them mathematically** using Dempster's Rule of Combination.

The total mass for each source must add up to 1:

- For E_1 → 0.6 + 0.4 = 1
- For E_2 → 0.7 + 0.3 = 1

Step4. Dempster's Rule of Combination

Now we combine mass from the two evidences E1 and E2: m1 and m2 into a new combined mass m12.

Formula:

$$m_{12}(A) = \frac{1}{1-K} \sum_{B \cap C = A} m_1(B) \times m_2(C)$$

Where:

- $m_{12}(A)$ = combined belief for subset A
- B and C are subsets from E_1 and E_2
- K = conflict factor
- $1 - K$ = normalization factor

Note: A → The **intersection** (common part) between B and C : A=B∩C

A is a *new subset* formed as the intersection of B and C.
It represents the **combined conclusion** that both evidences agree on.

Note:

Why normalization is needed

When you add up the beliefs from both evidences,
some of the total weight (the conflicting part) doesn't make sense because it's contradictory.

So, DST **removes this conflicting part (K)** and then **rescales the remaining beliefs** so that everything still adds up to **1** (i.e., 100% total belief).

Here:

- The numerator = combined agreement between evidence sets (non-conflicting parts).
- The denominator **(1 – K)** = *normalization factor* — it adjusts the total belief back to 1 after removing conflicts.

Note:

Imagine multiple **witnesses** describing the same suspect differently:

- Witness 1 says "A or C".
- Witness 2 says "C".

When we combine both — multiple pieces of support may point to "C".
The ∑ symbol ensures **you add all such supports together** before normalizing.

Here:

$$B = \{A, C\}, \quad C = \{D\}$$

So:

$$B \cap C = \emptyset$$

That means — there is **no overlap**, no common element between {A, C} and {D}.

which means **conflict**.

Here, the summation means:

Add up all the products of beliefs from both sources **where their intersection equals A**.

In simple words:

- We look at every possible pair of subsets **B** (from source 1) and **C** (from source 2).
- If both agree (their overlap = A), we multiply their beliefs and add it.

In our example, there's no summation because there are **no overlapping subsets** where both evidences agree on any possibility.

Step5. Conflict (K)

Conflict means "how much the evidence disagrees".

$$K = \sum_{B \cap C = \emptyset} m_1(B) \times m_2(C)$$

Here we check where the two evidences have **no overlap** (e.g., {A,C} ∩ {D} = ∅).

Calculate K:

$$K = m_1(A, C) \times m_2(D) = 0.6 \times 0.7 = 0.42$$

So, $K = 0.42$

Normalization factor = $1 - K = 0.58$

Step6. Combine Evidence

Note: Refer to Step3

Now calculate all possible intersections $B \cap C$.

B from E_1	C from E_2	Intersection (B∩C)	$m_1(B) \times m_2(C)$	Comment
{A,C}	{D}	∅	0.42	conflict
{A,C}	{A,C,D}	{A,C}	0.18	supports A or C
{A,C,D}	{D}	{D}	0.28	supports D
{A,C,D}	{A,C,D}	{A,C,D}	0.12	uncertainty

Now, ignore ***conflict*** and normalize:

Normalize using (1-K):

$$m_{12}(X) = \frac{m(X)}{1 - K}$$

Subset	Normalized m_{12}
{A,C}	0.18 / 0.58 = 0.31
{D}	0.28 / 0.58 = 0.48
{A,C,D}	0.12 / 0.58 = 0.21

All add up to 1 (0.31 + 0.48 + 0.21 = 1)

Step7. Final Interpretation

Outcome	Combined Belief (m_{12})	Meaning
{D}	**0.48**	Strongest belief that D is the murderer
{A,C}	**0.31**	Some belief that either A or C is the murderer
{A,C,D}	**0.21**	Remaining uncertainty (could be any)

Advantages of Dempster-Shafer Theory

1. Can handle incomplete information
Unlike Bayes, DST does **not require prior probabilities** for all outcomes.
You can assign belief to a set like {Rain, NoRain} to represent **uncertainty**.

Example:
Weather App: “Maybe rain, maybe not — I’m 20% unsure.”
→ DST can represent this explicitly.

2. Separates belief and uncertainty
DST distinguishes between:

- **Belief (Bel)** → how much the evidence supports something.

- **Plausibility (Pl)** → how much it could *possibly* be true.
  Thus, the **true probability** lies between these two.

Example:
Bel(Rain) = 0.6, Pl(Rain) = 0.8
→ So the actual probability of rain is **somewhere between 60% and 80%**.

This makes reasoning **transparent and cautious**.

3. Combines evidence from multiple sources
DST's **Dempster's Rule of Combination** lets you merge information from several sources — even if they are **partially uncertain or conflicting**.
Example:
- Satellite data → 70% sure of rain
- Human expert → 50% sure of rain
  DST fuses both into a combined, consistent belief.

4. Tolerant to conflicting evidence
When sources disagree, DST automatically **discounts conflicting parts** (through the conflict term K) and redistributes the rest proportionally.

You don't have to force a single probability estimate — it adapts.

Example:
Weather App says "Rain 80%,"
Old Farmer says "No Rain 70%."
DST can still give you a balanced belief after combining both opinions.

5. Represents ignorance explicitly
DST allows you to say:
"I have **no evidence** either way."

That's represented by giving belief to the **entire set of possibilities**, like {Rain, NoRain} — meaning "I'm completely unsure."

This is impossible in pure probability theory, where all probabilities must sum up strictly over defined outcomes.

10. Fuzzy Logic

Unlike classical logic which uses binary true or false values, **Fuzzy Logic** is a method of reasoning that **deals with degrees of truth** — not just "true" or "false". This is useful in systems that need to mimic human reasoning, such as in control systems for appliances or expert systems.

The Basic Idea

In **classical logic**, everything is **black or white** —
- It's either **true or false**,
- **yes or no**,
- **0 or 1**.

Example:

- Is the room *hot*?
  - If temperature > 30°C → **True (1)**
  - Else → **False (0)**

But real life isn't that simple —
What about **29°C** or **28.5°C**? It's *somewhat* hot, right?

That's where **Fuzzy Logic** comes in!

What Fuzzy Logic Says

"Things can be **partially true**."
It deals with **degrees of truth**, between **0 and 1**.

Truth value	Meaning
0.0	Completely False
0.5	Half True
1.0	Completely True

So instead of saying "Hot or Not Hot,"
we say **"How hot is it?"**

Real-life Example — "Temperature"

Imagine you're setting an air conditioner.

- If it's **10°C**, it's clearly *cold*
- If it's **40°C**, it's *hot*
- But what about **25°C**?
  → It's not *completely cold*, not *completely hot* — it's *somewhat warm.*
- And what about **50°C**?
  → It's not just *hot* — it's *somewhat too hot.*

Fuzzy Logic allows us to **express this "somewhat"** mathematically.

Imagine an Air Conditioner with Fuzzy Logic

Temperature	Degree of "Hotness"
10°C	0.0 (Not hot)
25°C	0.4 (A little hot)
35°C	0.8 (Quite hot)
45°C	1.0 (Very hot)

The AC doesn't just turn ON or OFF —
It adjusts **fan speed gradually**, depending on *how hot* the room is.

Classical Logic Vs Fuzzy Logic

Temperature	Classical (Boolean)	Fuzzy (Degree of Truth)
10°C	Cold = 1, Hot = 0	Cold = 1.0, Warm = 0.0, Hot = 0.0
25°C	Cold = 0, Hot = 1 ? (unclear)	Cold = 0.3, Warm = 0.8, Hot = 0.2
40°C	Cold = 0, Hot = 1	Cold = 0.0, Warm = 0.1, Hot = 1.0

So, at 25°C, the system "knows": It's **30% Cold**, **80% Warm**, and **20% Hot** — all at the same time.

How It Works (Steps) – Smart Air Conditioner Example

1. **Fuzzification**
2. **Rule Evaluation (Inference)**
3. **Aggregation**
4. **Defuzzification**

Let us understand the above four terms in detail:

Fuzzification

Convert crisp (exact) input into fuzzy (inexact, degree-based) values.

You measure the room temperature: **31°C**
Now, instead of saying "It's Hot" or "It's not Hot,"
the fuzzy system says:

- "Hot" = 0.6
- "Warm" = 0.4
- "Cold" = 0.0

Rule Evaluation (Inference)

*Apply fuzzy **IF–THEN** rules (like expert system rules).*

Say, the following are Rule1, Rule2 and Rule3:

```
IF temperature IS Cold THEN fan_speed IS Low
IF temperature IS Warm THEN fan_speed IS Medium
IF temperature IS Hot THEN fan_speed IS High
```

At 31°C, we found:

- Hot = 0.6
- Warm = 0.4
- Cold = 0.0

So, rules 2 and 3 **fire partially** (with their degrees):

Rule	Temperature Condition	Firing Strength	Fan Speed Output
1	Cold	0.0	Low
2	Warm	0.4	Medium
3	Hot	0.6	High

Meaning:

- The “Warm” rule says fan should be **Medium (40%)**
- The “Hot” rule says fan should be **High (60%)**

Aggregation

*Combine all rules’ fuzzy outputs into **one single fuzzy result**.*

Output Variable	Membership Strength	Contribution
Low	0.0	none
Medium	0.4	moderate
High	0.6	strong

They are **merged (aggregated)** into one overall fuzzy output:

- 40% membership of *Medium*
- 60% membership of *High*

This fuzzy combination represents: “Fan speed should be somewhere between Medium and High.”

Defuzzification

Convert the fuzzy output into a crisp, real number (e.g., fan speed = 75%).

Example:

After combining all fuzzy outputs:

Fan Speed	Degree
50% (Medium)	0.4
80% (High)	0.6

$$\text{Weighted Average} = \frac{\sum(w_i \times x_i)}{\sum w_i}$$

Where:

- x_i = the individual values (for example, each fan speed)
- w_i = the corresponding weights (for example, the degree of membership)
- $\sum$ = summation (add them all up)

Weighted average:

$$\text{Fan Speed} = \frac{(50 \times 0.4) + (80 \times 0.6)}{0.4 + 0.6} = 68\%$$

Multiply each value (fan speed) by its weight, then divide by the total of the weights.

So, the fan runs at **≈70% speed** — not fully ON, not too low.

Without Fuzzy Logic:

- IF temperature ≥ 30°C → turn fan to full speed (100%)
- ELSE → fan off (0%)

→ Very uncomfortable! Fan keeps jumping ON/OFF.

With Fuzzy Logic:

- At 25°C → Fan = 60%
- At 35°C → Fan = 90%
- Smooth, intelligent adjustment — like a human decision.

Fuzzy Sets

In fuzzy logic, elements can **partially belong** to a set.

Example:

Fuzzy set *Hot* could be defined as:

- 25°C → 0.3 (somewhat hot)
- 30°C → 0.7 (quite hot)
- 40°C → 1.0 (definitely hot)

So the **fuzzy set "Hot"** is represented mathematically as:

$$\text{Hot} = \{(25, 0.3), (30, 0.7), (40, 1.0)\}$$

Each pair is (temperature, degree of membership).

Fuzzy Operations

In a fuzzy rule base, these operations are used for rule conditions:

```
IF temperature IS Hot AND humidity IS High THEN fan_speed IS High
```

The fuzzy "AND" combines two input memberships (Hot, High humidity).

Just like in **classical set theory**, we can perform operations such as:

- **AND** (∩) → Intersection
- **OR** (∪) → Union
- **NOT** (¬) → Complement

But in **fuzzy logic**, since elements have **degrees of membership (0–1)**

So, Fuzzy operations allow a system to:

- Merge multiple fuzzy inputs
- Compare them logically (AND / OR / NOT)
- Decide the overall fuzzy output

Example

Let's say we have two fuzzy sets:

x (temperature)	μA(x) = "Warm"	μB(x) = "Hot"
20°C	0.4	0.0
25°C	0.8	0.3
30°C	0.3	0.7
35°C	0.0	1.0

Each μ means *membership value* (degree of truth).

Fuzzy AND (Intersection)

Classical: A∩B
Fuzzy: Take the **minimum** of the two memberships.

$$\mu_{A\cap B}(x) = \min(\mu_A(x), \mu_B(x))$$

Example:

x	Warm	Hot	min(Warm, Hot)
20°C	0.4	0.0	**0.0**
25°C	0.8	0.3	**0.3**
30°C	0.3	0.7	**0.3**
35°C	0.0	1.0	**0.0**

Fuzzy OR (Union)

Classical: A∪B
Fuzzy: Take the **maximum** of the two memberships.

$$\mu_{A \cup B}(x) = \max(\mu_A(x), \mu_B(x))$$

Example:

x	Warm	Hot	max(Warm, Hot)
20°C	0.4	0.0	**0.4**
25°C	0.8	0.3	**0.8**
30°C	0.3	0.7	**0.7**
35°C	0.0	1.0	**1.0**

Fuzzy NOT (Compliment)

Classical: ¬A
Fuzzy: Take **1 – membership value**.

$$\mu_{\neg A}(x) = 1 - \mu_A(x)$$

Example:

x	Warm	NOT Warm
20°C	0.4	**0.6**
25°C	0.8	**0.2**
30°C	0.3	**0.7**
35°C	0.0	**1.0**

When to use which operation

Fuzzy Operation	Purpose (Why/When We Use It)
AND (min)	To find how strongly *all* conditions are true together.
OR (max)	To find how strongly *at least one* condition is true.
NOT (1–μ)	To find how strongly a condition is *not true*.

Advantages

- Handles uncertainty & vagueness naturally
- More human-like reasoning
- Works well with imprecise sensors
- Robust and easy to interpret

Applications of Fuzzy Logic

Domain	Example
Home Appliances	ACs, washing machines (LG, Samsung use fuzzy control)
Automotive	Automatic gear shift, anti-lock braking systems
Healthcare	Diagnosis support (e.g., fuzzy rules for symptom strength)
Finance	Risk assessment ("investment is somewhat risky")
Robotics	Balancing, obstacle avoidance decisions

Fuzzy Logic Vs Certainty Factor (CF)

Feature	Fuzzy Logic	Certainty Factor (CF)
Purpose	Helps computers understand **how much** something is true.	Helps computers understand **how sure** we are about something being true
Meaning of Degree (μ)	Degree of **membership** — how much a value belongs to a fuzzy set	Degree of **belief or confidence** — how sure we are about a hypothesis
Range	$\mu \in [0, 1]$	CF ∈ [−1, +1] (+ means belief, − means disbelief)
Example Question	"How hot is 30°C?"	"How sure are we that the battery is dead?"
Interpretation of Degree	$\mu = 0.7$ → "30°C is **0.7 hot** (70% hot)"	CF = 0.7 → "I'm **70% confident** the battery is dead"
Used in	Fuzzy systems (e.g., Air Conditioning, Fan control, Washing machine)	Expert systems (e.g., Medical diagnosis, Car fault detection)

Program: Creating a Fuzzy Air Conditioner System in Python

Note:

Python is better for fuzzy logic systems.
Prolog is good for *symbolic* logic, not for *gradual numeric reasoning.*

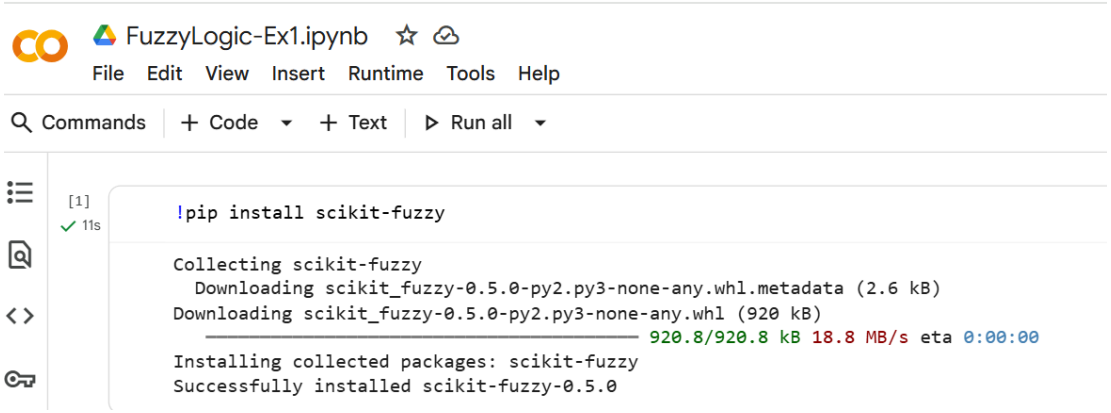
Requirement:

Let's say you want:

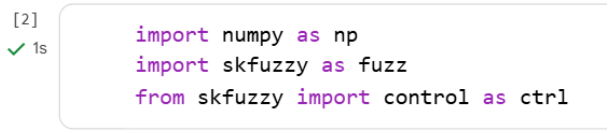
- **Inputs:** Temperature (°C), Humidity (%)
- **Output:** Fan Speed (%)

Step1. Open Google Colab, create a new Jupyter Notebook file, connect to the Server's CPU

Step2. Install the fuzzy library



Step3. Write the necessary imports



Step4. Define the input and output universe (domains)

```
# Input variables
temperature = ctrl.Antecedent(np.arange(0, 41, 1), 'temperature')
#creates a fuzzy input variable named "temperature" whose possible crisp values are 0-40°C.

humidity = ctrl.Antecedent(np.arange(0, 101, 1), 'humidity')
```

```
# Output variable
fan_speed = ctrl.Consequent(np.arange(0, 101, 1), 'fan_speed')
```

Step5. Define the fuzzy membership functions for inputs and outputs (the **fuzzification** setup stage)

```
#Define Fuzzy membership functions for input and output (Fuzzy sets)
temperature['cold'] = fuzz.trimf(temperature.universe, [0, 0, 20])
#trimf(): creates a triangular membership function
temperature['warm'] = fuzz.trimf(temperature.universe, [15, 25, 35])
#defines Warm peaking at 25°C and zero near 15 and 35.
#Three-element vector controlling shape of triangular function. Requires a <= b <= c.
temperature['hot']  = fuzz.trimf(temperature.universe, [25, 40, 40])

humidity['low'] = fuzz.trimf(humidity.universe, [0, 0, 50])
humidity['high'] = fuzz.trimf(humidity.universe, [40, 100, 100])

fan_speed['low']    = fuzz.trimf(fan_speed.universe, [0, 0, 50])
fan_speed['medium'] = fuzz.trimf(fan_speed.universe, [25, 50, 75])
fan_speed['high']   = fuzz.trimf(fan_speed.universe, [50, 100, 100])
```

Eg:
In the line:
temperature['warm'] = fuzz.trimf(temperature.universe, [15, 25, 35])

- “Warm” peaks at 25°C, decreases near 15°C and 35°C.

- [15, 25, 35] means:
  - Below 15°C → not warm
  - 25°C → fully warm (membership = 1)
  - Above 35°C → not warm

25°C is the perfect "warm" feeling

That is **fuzzification** — converting a *crisp value (30°C)* into *fuzzy degrees of membership*.

Step6. Define Fuzzy Rules **(Rule Evaluation)**

```
rule1 = ctrl.Rule(temperature['hot'] & humidity['high'], fan_speed['high'])
#ctrl.Rule(condition, action). creates a fuzzy rule object that links input conditions to an output fuzzy set.
rule2 = ctrl.Rule(temperature['warm'] & humidity['low'], fan_speed['medium'])
rule3 = ctrl.Rule(temperature['cold'], fan_speed['low'])
```

Step7. Build the Control System **(Aggregation) – Creating a Fuzzy Inference System**

```
ac_ctrl = ctrl.ControlSystem([rule1, rule2, rule3])
#creates a fuzzy control system that contains the rule base and variable definitions;
#this object arranges how rules are combined and how outputs are aggregated.
```

Step8. Build the simulation

```
ac_sim = ctrl.ControlSystemSimulation(ac_ctrl)
#constructs a runtime simulation object where you can insert crisp input values,
#run the fuzzy inference (fuzzify → evaluate rules → aggregate → defuzzify), and then read outputs.
```

The above code creates a simulation object

Step9. Give Input (sensor readings) and Computer Output

```
ac_sim.input['temperature'] = 30  # Celsius
ac_sim.input['humidity']    = 60  # Percent

ac_sim.compute()
```

ac_sim.compute() runs the fuzzy inference cycle including "Defuzzification":

1. **Fuzzification:** compute membership degrees μ for each relevant fuzzy set.
2. **Rule evaluation:** compute firing strengths for each rule (AND/OR applies).
3. **Implication / clipping:** produce output fuzzy sets scaled by rule strengths.
4. **Aggregation:** combine all output fuzzy sets into a single fuzzy output distribution (for fan_speed).
5. **Defuzzification:** convert aggregated fuzzy output into a single crisp number (default method is centroid).

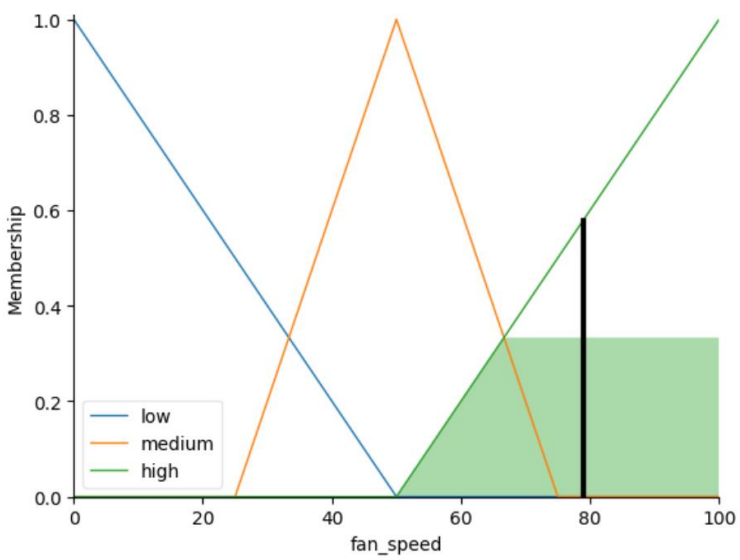
Step10. Read and display the result

```
print("Fan speed = {:.2f}%".format(ac_sim.output['fan_speed']))
#ac_sim.output['fan_speed'] contains the defuzzified crisp value computed during compute().
#We format and print it with two decimal places to show the percentage fan speed.
```

```
Fan speed = 78.89%
```

Step11. Plotting the fan speed

```
fan_speed.view(sim=ac_sim)
```



fan_speed.view(sim=ac_sim): draws a plot of the fan_speed fuzzy membership functions and overlays the aggregated fuzzy output and the defuzzified crisp value — useful for visualizing how the final number was obtained.

Note: Interpreting the chart:

The height of the shaded area (around **0.35**) means:
"I am about 35% confident that the fan speed should be high."
This height **cuts the original triangle** at 0.35 and shades everything under it.

Program: Creating a Fuzzy Car Braking System in Python

Note:

Requirement:

Decide **brake force** (Low / Medium / High) based on:

- Distance to obstacle (in meters)
- Car speed (in km/h)

"Automatic Car Braking System" based on **distance to obstacle** and **vehicle speed**.

Step1. Open Google Colab, create a new Jupyter Notebook file, connect to the Server's CPU

Step2. Install the fuzzy library as done in the previous program

Step3. Write the necessary imports

```
[2]
✓ 1s
```

```
import numpy as np
import skfuzzy as fuzz
from skfuzzy import control as ctrl
```

Step4. Define the input and output universe (domains)

```
# Input variables
distance = ctrl.Antecedent(np.arange(0, 101, 1), 'distance')
#np.arange(0, 101, 1) means values from 0 to 100 with step = 1.
speed = ctrl.Antecedent(np.arange(0, 121, 1), 'speed')
```

```
# Output variable
brake = ctrl.Consequent(np.arange(0, 11, 1), 'brake')
#Brake force between 0-10 units (you can think of it as intensity or percentage)
```

Step5. Define the fuzzy membership functions for inputs and outputs

```
#Define Fuzzy membership functions for input and output (Fuzzy sets)
distance['close'] = fuzz.trapmf(distance.universe, [0, 0, 20, 40])
distance['medium'] = fuzz.trimf(distance.universe, [30, 50, 70])
distance['far'] = fuzz.trapmf(distance.universe, [60, 80, 100, 100])

#trapmf = trapezoidal membership function
#trimf = triangular membership function

speed['slow'] = fuzz.trapmf(speed.universe, [0, 0, 30, 50])
speed['moderate'] = fuzz.trimf(speed.universe, [40, 70, 100])
speed['fast'] = fuzz.trapmf(speed.universe, [90, 110, 120, 120])

brake['low'] = fuzz.trapmf(brake.universe, [0, 0, 2, 4])
brake['medium'] = fuzz.trimf(brake.universe, [3, 5, 7])
brake['high'] = fuzz.trapmf(brake.universe, [6, 8, 10, 10])
```

Use Triangular when:

- There's a **clear single ideal value** (e.g., "Best Speed = 60 km/h")
- You want smooth overlap between adjacent sets.

Use Trapezoidal when:

- You have a **range of equally valid values** (e.g., "Safe Zone = 40–60 km/h")
- You want the system to be more tolerant or stable over a range.

Feature	Triangular (trimf)	Trapezoidal (trapmf)
Shape	▲ Triangle	■ Trapezoid (flat top)
Parameters	3 points → [a, b, c]	4 points → [a, b, c, d]

Feature	Triangular (trimf)	Trapezoidal (trapmf)
Peak Value (μ = 1)	Only at one point (b)	Flat region between b and c
Smoothness	Sharp peak — one value is "fully true"	Plateau — multiple values are "equally true"
Usage	When the concept has a *single precise center*	When the concept has a *range of equally strong truth*
Example	"Warm temperature"	"Comfortable temperature"

Step6. Define Fuzzy Rules

```
rule1 = ctrl.Rule(distance['close'] & speed['fast'], brake['high'])
rule2 = ctrl.Rule(distance['close'] & speed['moderate'], brake['high'])
rule3 = ctrl.Rule(distance['medium'] & speed['moderate'], brake['medium'])
rule4 = ctrl.Rule(distance['far'] & speed['slow'], brake['low'])
rule5 = ctrl.Rule(distance['far'] & speed['fast'], brake['medium'])
```

Step7. Build the Control System

```
brake_ctrl = ctrl.ControlSystem([rule1, rule2, rule3, rule4, rule5])
```

Step8. Build the simulation

```
brake_sim = ctrl.ControlSystemSimulation(brake_ctrl)
```

Step9. Give Input (sensor readings) and Computer Output

```
brake_sim.input['distance'] = 25
brake_sim.input['speed'] = 90

brake_sim.compute()
```

Step10. Read and display the result

```
print(f"🚗 Recommended brake force: {brake_sim.output['brake']:.2f}")
```

```
🚗 Recommended brake force: 8.16
```

Step11. Plotting the break intensity

```
brake.view(sim=brake_sim)
```